

RENTGEN: BEHAVIOR-FIRST API TESTING BEFORE AUTOMATION

A DETERMINISTIC METHOD FOR GENERATING STRUCTURED API EVIDENCE FROM A SINGLE KNOWN-GOOD REQUEST

BY LIUDAS JANKAUSKAS

[First version: July 2026]
[rentgen.io]
pdf-version: [PDF]
www-version: [WWW]

ABSTRACT

API testing often begins with a single successful request, which is frequently misinterpreted as sufficient proof that an endpoint works. While this confirms basic reachability, it reveals little about how the API handles malformed input, boundary values, authentication failures, protocol edge cases, or performance characteristics.

Rentgen is a local-first, behavior-first API testing tool designed to fill this gap. From one known-good request, it automatically generates structured mutations and protocol checks, executes them deterministically, and produces clear, configurable observations about the API’s real behavior. The resulting project is portable and can be reused across the desktop application, CLI, Docker, and CI/CD environments.

Rentgen does not aim to prove functional correctness or replace domain-specific tests and regression suites. Instead, it delivers fast, actionable insights during the critical phase before traditional automation is written — enabling teams to discover risks and make better decisions earlier.

This paper describes Rentgen’s operating model, test taxonomy, deterministic evaluation rules, and field observations from real-world production APIs, while also outlining an experimental AI-directed discovery architecture.

INTRODUCTION

Modern API clients make it trivial to send a request and receive a successful response. However, a single 2xx response reveals very little about how the API actually behaves when its assumptions are challenged — malformed input, boundary values, missing or optional fields, authorization failures, unsupported methods, oversized payloads, or missing security protections.

Traditional test automation can uncover these issues, but it demands significant upfront investment: designing scenarios, building fixtures, writing assertions, and maintaining the resulting suite as the API evolves. This creates a practical gap between the first working request and the first meaningful, maintainable test coverage.

Rentgen was designed to fill this gap through the *Automation Before Automation (ABA)* approach [2]. It transforms a single known-good request into a structured set of controlled variations, executes them deterministically, and reveals how the API actually behaves — all before any traditional test suite is written.

The goal is not to replace existing automation frameworks, but to provide fast, actionable insights early enough for teams to discover risks and make better-informed decisions about their testing strategy.

1. THE TESTING GAP

A single 2xx response shows that one specific request succeeded under one specific set of conditions. It does not establish how the API behaves when those conditions change or its assumptions are violated. When malformed or invalid input produces a 5xx response, the server has encountered an unexpected failure rather than rejecting the request in a controlled manner [1].

The real gap in API development is not the lack of tools for sending requests. It lies between sending a quick request and obtaining **broad, reviewable evidence** about the API’s actual behavior — without first building and maintaining a complete test suite.

Well-designed automated test suites are extremely valuable because they encode product-specific expectations. However, this specificity creates a significant upfront cost: engineers must define scenarios, set up fixtures, write assertions, manage test data, and maintain the suite as the API evolves. As a result, easily detectable protocol, validation, and security issues can survive until later — and more expensive — stages of development.

Automation Before Automation (ABA) [2] defines a distinct early discovery phase: using automated mechanisms to generate observations and failure signals *before* stable regression tests are designed. This approach does not compete with traditional automation — it complements it by helping teams identify which behaviors and risks deserve long-term testing investment.

Phase	Primary question	Typical artifact
Manual request	Can I reach the endpoint?	A known-good cURL or equivalent request
Rentgen / ABA	How does it behave when assumptions are disturbed?	Generated observations, findings, and project expectations
Scripted automation	Which agreed behaviors must remain stable?	Maintained assertions and regression suites
CI/CD gate	Should this build proceed?	Repeatable policy decision and report

2. DESIGN PRINCIPLES

Rentgen follows five design principles that define how tests are generated, executed, evaluated, and stored.

- **Seed-based test generation.** A known-good request is used as the input for generating test variations. It is treated as a starting point, not as a complete API specification.
- **Deterministic test execution.** Request mutations, protocol checks, and evaluation rules are produced and executed by a deterministic engine. API responses are recorded as runtime observations and may vary depending on the state of the tested system and environment.
- **Configurable expectations.** Default expected results can be reviewed, modified, ignored, or re-enabled by the user. These decisions are stored as part of the project configuration.
- **Human review.** Rentgen identifies differences between expected and observed behavior. These findings require contextual review and do not automatically prove the existence of a defect or vulnerability.
- **Local execution.** The application and test engine run on the user’s machine. Requests are sent only to the configured target API and are not uploaded to a Rentgen backend, telemetry service, or external analytics platform.

These principles allow Rentgen to generate repeatable API tests while keeping the test configuration explicit, portable, and under the user’s control.

3. OPERATING MODEL

The workflow starts with a request that is known to produce a successful response. In the desktop application, the request can be imported directly from cURL format and parsed into its components: HTTP method, URL, headers, query parameters, body, and authentication headers. Project variables can then be resolved using the selected environment. During data-driven execution, the original request is executed first to establish a baseline for the generated variations.

Field mapping

Request fields are classified into test-relevant categories such as string, number, numeric string, email, phone, URL, IPv4, enum, bounded range, random value, or do-not-test. These mappings can be inferred by Rentgen and adjusted by the user. This allows the engine to apply generic mutation generators while the user retains control over field semantics and expected behavior.

Generation, execution, and evaluation

Once field mappings are defined, the deterministic engine generates test variations, sends the resulting requests to the target environment, evaluates the responses, and stores the results.

Stage	Deterministic responsibility
Generate	Create field mutations and protocol-level variants based on stored mappings and enabled checks.

Execute	Send each generated request and capture its status code, headers, body, size, and timing information.
Evaluate	Compare the observed response with the configured expectation: 2xx for accepted input or 4xx for controlled rejection.
Report	Present pass, fail, manual-review, ignored, and potential-bug results together with the corresponding request and response evidence. Disabled checks remain visible.
Persist	Store mappings, expectations, ignored tests, collections, variables, and project state in a portable .rentgen project file.

By default, Rentgen expects valid test data to produce a 2xx response and invalid test data to produce a controlled 4xx response. A 5xx response is treated as a server-side failure signal. These defaults are heuristics rather than universal correctness rules. Starting with Rentgen v1.22, users can change the expected outcome between 2xx and 4xx for individual data-driven test cases, and these expectations are stored in the project.

4. TEST TAXONOMY

Rentgen organizes its generated results into three broad categories: data-driven input tests, protocol and security-hygiene checks, and performance insights.

Data-driven input tests

These tests modify one field or structural property at a time while keeping the rest of the seed request intact. The goal is not random fuzzing, but targeted mutation based on the field mappings and validation assumptions defined in the project.

Category	Example mutation	Signal sought
Presence	Remove a field or send null	Required/optional behavior and controlled rejection
Representation	Replace a number with a string or object	Type enforcement and parser resilience
Boundary	Use min-1, min, max, and max+1	Off-by-one behavior and range enforcement
Normalization	Add leading/trailing whitespace or change case	Canonicalization and unintended data modification
Domain format	Send an invalid email, IPv4 address, URL, or numeric string	Field-specific validation behavior
Size	Send an oversized field or request body	Payload-size controls and graceful rejection
Structure	Send malformed JSON or an unexpected data structure	Parser stability and error handling

Protocol and security-hygiene checks

In addition to data mutations, Rentgen performs observable protocol and security-hygiene checks covering authentication handling, HTTP methods, route behavior, payload handling, cross-origin behavior, reflected content, and response headers. These checks overlap with several OWASP API Security concerns but remain screening-level checks rather than a penetration test [\[2\]](#) [\[3\]](#).

Check family	Checks and illustrative expectations	Interpretation boundary
Authentication	Missing or invalid credentials produce 401	Different status codes may be intentional; business-level authorization is not verified.
Methods and discovery	OPTIONS returns 200/204 with an Allow header; unsupported methods produce 405 or 501	Gateway or proxy policies may change the observed response.
Routing	Unknown routes produce 404; uppercase domain and path variations are handled consistently	Framework or security policy may intentionally hide resources.
Payload handling	Oversized requests produce 413 and malformed payloads are rejected without a server failure	Infrastructure may enforce limits before the application layer.
Cross-origin and reflected content	CORS behavior is examined and submitted payload content is checked for unsafe reflection	Expected behavior depends on the API's intended consumers and response format.
Response headers	Server-version exposure, HSTS, clickjacking protection, X-Content-Type-Options, and cache-control are checked	Applicability depends on the response type and deployment context.

Performance insights

Rentgen provides performance observations rather than capacity certification. It records response timing across generated tests and includes a configurable concurrent load test that reports average response time, p50, p90, p95, and the number of observed 4xx and 5xx responses. Additional checks cover response size, host ping latency, the estimated share of response time attributable to network latency, and GET responses containing JSON arrays when the request URL contains no query parameters that would indicate pagination or limiting. These results identify latency, payload-size, stability, and pagination signals that may warrant dedicated performance testing.

INTERPRETATION LIMIT: The generated load test provides timing and failure signals under the configured request count and concurrency. It does not establish production capacity, saturation limits, or service-level compliance without a controlled workload model and representative test environment.

5. PORTABLE PROJECT CONFIGURATION

A .rentgen project file stores the configuration and state associated with a Rentgen project: requests, collections, static and dynamic environment variables, field mappings, expected outcomes, ignored checks, and execution history. The stored mappings, expectations, and enabled checks allow Rentgen to regenerate and evaluate the same intended test cases across repeated runs, while the observed responses continue to reflect the current state of the target API.

The project can be shared or maintained in team-controlled storage without relying on a hosted synchronization service. When project files contain authentication credentials or other sensitive environment values, they must be protected or sanitized before being committed to source control.

The desktop application provides interactive project configuration and result review. The same project can be executed through the CLI with collection and environment selection and machine-readable JSON reporting. This allows tests configured in the desktop application to run through the CLI, Docker, or CI/CD pipelines without being rewritten in another testing framework [\[5\]](#).

Surface	Role	Typical use
Desktop application	Interactive configuration and review	Import requests, adjust field mappings, configure expectations, and inspect results
CLI	Headless execution	Run a project or selected collection from a terminal or script
Docker	Containerized execution	Run the Rentgen engine in a controlled container environment

CI/CD	Automated execution and gating	Generate reports and fail pipelines according to configured evaluation rules
-------	--------------------------------	--

6. LOCAL-FIRST TRUST MODEL

API requests often contain credentials, personal or regulated data, internal hostnames, and proprietary schemas. Rentgen treats locality as an architectural constraint: the desktop application does not require user accounts, hosted synchronization, telemetry, or a Rentgen-operated cloud execution service. Apart from sending requests to the target API selected by the operator, Rentgen does not transmit requests, responses, projects, or generated results to Rentgen infrastructure. These artifacts remain on the operator's machine. This allows Rentgen to be used in environments where SaaS API testing tools or external AI services are restricted.

Local execution does not eliminate operational risk. The operator remains responsible for endpoint authorization, test-data handling, credential storage, possible side effects, traffic volume, and the security of the host machine and project files. Generated tests must only be executed against systems the operator is authorized to test. Endpoints that modify or delete data, initiate transactions, or produce other consequential effects require additional safeguards beyond default test generation.

7. EVIDENCE AND FIELD OBSERVATIONS

Rentgen has been tested against public production APIs including ChatGPT, Amazon, Jira, Proton Pass, Revolut, Trello, Azure DevOps, and Stripe. Published case studies document observations such as malformed-input 500 responses, late payload rejection, HTML returned from JSON endpoints, unexpected method handling, authorization inconsistencies, missing response protections, and elevated latency. In the ChatGPT case, responsible disclosure led to fixes by OpenAI [\[6\]](#).

These cases demonstrate the feasibility of the approach and Rentgen's ability to surface concrete production signals. However, they do not constitute a controlled comparative benchmark. Different endpoints, configurations, time periods, authorization contexts, and generated test sets were used. The results should therefore be interpreted as field observations rather than general assessments of any API or vendor.

Public API	Generated Tests	Observed Signals	Reported Follow-up
ChatGPT	200+ generated tests	A 10 MB request produced 500 instead of controlled payload rejection; permissive CORS behavior was observed; invalid field types were accepted with 200 OK	OpenAI fixed several findings
Amazon	100+ generated tests	Missing authentication produced 400 instead of 401; an uppercase path returned a CloudFront HTML error page from a JSON API; an approximately 10 MB request was processed before returning 404 instead of an early 413 rejection	No vendor follow-up documented
Proton Pass	100+ generated tests	An approximately 10 MB request returned 400 after body parsing instead of an early 413 rejection; missing authentication material initially produced 400 before 401 was returned with the required headers present; a non-existent route returned 405 even though the response body indicated that the route was not found	No vendor follow-up documented
Jira	100+ generated tests	An unsupported HTTP method returned 403 instead of 405 or 501; a non-existent route returned 405 instead of 404; an approximately 10 MB request was fully processed before returning 400 instead of an early 413 rejection	No vendor follow-up documented
Revolut	100+ generated tests	The amount field accepted a numeric string with 200 OK, while malformed strings and boolean values produced 500 errors; invalid types supplied for wallet identifiers also produced 500 errors; unsupported enum values caused additional 500 responses	No vendor follow-up documented
Trello	100+ generated tests	Malformed values for the desc field produced 500 errors; a null card name was accepted with 200 OK and created a card with an empty title; the isTemplate boolean field accepted a string value and created the card	No vendor follow-up documented
Azure DevOps	100+ generated tests	An empty displayName produced a 500 error; boolean and numeric values were accepted and caused the field to be stored as null; an approximately 10 MB request produced 500 instead of a controlled 413 rejection	No vendor follow-up documented
Stripe	100+ generated tests	An unsupported HTTP method returned 403 with an HTML nginx response instead of a structured 405 response; the email field accepted multiple syntactically invalid address formats	No vendor follow-up documented

8. RELATIONSHIP TO EXISTING TESTING

Rentgen does not replace API clients such as Postman or cURL, nor programmable test frameworks such as REST Assured, Playwright, or contract-testing tools. It is also not intended to replace dedicated security scanners or load-testing systems. These tools address different phases, depths, and objectives of testing.

Most importantly, Rentgen is not designed to model product-specific happy paths, multi-step workflows, business rules, or stateful user scenarios. Those belong in maintainable, programmable test suites. The absence of such capabilities is not an unfinished feature or a limitation to be removed; it is an intentional boundary of the tool.

Rentgen operates in the interval after a request first works but before a maintained test model exists. Starting from one known-good request, it systematically explores variations and records observable behavior, exposing validation gaps, protocol inconsistencies, error-handling weaknesses, and early performance or security signals. These observations help teams decide what must be fixed, accepted, investigated further, or encoded as durable automation.

Evaluating Rentgen as a replacement for a complete test framework is therefore a category error. Its purpose is not to automate everything a team already knows must work. Its purpose is to rapidly expose behaviors and test conditions that the team may not yet know it should be testing.

The intended workflow is cumulative:

Build or change an endpoint → Run Rentgen for discovery → Review and triage observations → Encode valuable business and regression checks in maintainable automation → Enforce them through CI/CD

This relationship aligns with broader secure development practices, such as those described in NIST's Secure Software Development Framework, which treats verification as part of a larger engineering discipline rather than the outcome of a single tool [\[4\]](#).

9. LIMITATIONS AND VALIDITY CONSIDERATIONS

Rentgen produces structured and repeatable evidence. As with any black-box testing approach, the resulting coverage depends on the supplied requests, observable responses, configured environment, and available domain context.

Limitation	Consequence	Mitigation
Input scope	Generated exploration follows the supplied requests, mapped fields, and reachable endpoints.	Use representative requests and collections covering different parts of the API.
Domain context	Generic checks reveal observable behavior but do not determine product-specific business correctness.	Use the findings to guide dedicated business and regression assertions where needed.
State-changing	Tests against mutation endpoints may create data or trigger	Use controlled environments, disposable data, and

operations	workflows.	explicit test exclusions.
Infrastructure influence	Gateways, WAFs, proxies, and caches may affect the responses observed by the tool.	Record the environment and inspect the captured request and response evidence.
Context-dependent findings	Some detected inconsistencies may reflect intentional API or infrastructure decisions.	Review findings in context and ignore or re-enable individual checks as appropriate.
Case-study generalizability	Published observations represent specific endpoints, configurations, and points in time.	Treat them as evidence of detection capability rather than comparative product benchmarks.

These boundaries define how Rentgen evidence should be interpreted; they do not reduce its value as an early, systematic API discovery layer.

10. AI-DIRECTED API DISCOVERY WITH RENTGEN

Rentgen.io is researching an AI-directed extension of Rentgen in which a language model participates directly in a live API discovery session. This direction differs fundamentally from workflows that use an LLM to generate test cases or automation code for later execution. The model does not produce a test suite. It decides what Rentgen should send next.

The interaction begins with the same minimal input already used by Rentgen: one working cURL request. Rentgen executes the request and returns a structured observation containing the response, request history, and detected signals. The language model then selects the next API interaction: it may change a field, value, type, header, method, path, payload structure, or another request property. Rentgen executes the selected request, captures what happened, and returns the new observation to the model.

One working cURL → Rentgen executes the request → LLM observes the result → LLM decides what to send next → Rentgen executes again → The cycle continues → Rentgen produces an evidence-backed discovery report

Each response can change the direction of the exploration. An unexpected 2xx response, a server-side error, reflected input, inconsistent validation, a changed response structure, or an unusual timing signal may cause the model to investigate that behavior further. The exploration path is therefore created during execution rather than predefined before the session begins.

NOT TEST GENERATION: The LLM does not write tests for somebody or something else to execute later. It makes runtime testing decisions, Rentgen sends the requests, and each real response informs the next decision.

Boruzele provides a working proof of concept for this runtime decision loop. It observes the current UI, asks a language model to choose the next interaction, executes that interaction through Playwright, collects failure signals, and returns the updated state for the next decision. It does not generate test cases or a maintainable automated suite [8].

The proposed Rentgen extension applies the same principle to APIs. In this setting, the control loop is potentially simpler and more constrained: the action is an API request, the observation is an API response, and both can be captured precisely. Rentgen already provides the request execution, evidence collection, deterministic checks, and reporting foundation. The language model would add adaptive exploratory decision-making on top of that foundation.

The intended experience is straightforward: provide any working cURL request and receive a report produced through autonomous, response-driven API exploration. This would extend Automation Before Automation from systematic predefined variation to adaptive discovery, allowing the system to investigate behaviors that were not explicitly encoded in advance [2].

This remains a Rentgen.io research direction rather than a capability of the current Rentgen release. It would complement, not replace, the existing deterministic engine: deterministic checks provide repeatability and explicit expectations, while AI-directed exploration can search for unexpected paths and decide dynamically where deeper investigation may be valuable.

CONCLUSION

Rentgen demonstrates that useful API testing can begin before a traditional automated test suite exists. Starting from one known-good request, it systematically executes request variations and captures evidence about validation, robustness, protocol behavior, security hygiene, and performance. This makes the interval between a working endpoint and maintained automation an active discovery stage rather than an untested gap.

As an implementation of Automation Before Automation, Rentgen helps teams discover what deserves investigation, correction, or later regression coverage. Its current deterministic engine makes this discovery repeatable and portable across desktop, CLI, Docker, and CI/CD. The proposed AI-directed extension would add adaptive exploration: an LLM decides what request to try next from the observed response, while Rentgen remains responsible for execution and evidence. The result is a distinct progression from one working cURL to systematic discovery today and autonomous, response-driven API exploration in the future.

[Rentgen: rentgen.io]
 [Liudas Jankauskas homepage: gaontime.com]
 [More research: gaontime.com/research]

REFERENCES

- [1] R. Fielding, M. Nottingham, and J. Reschke, HTTP Semantics, RFC 9110, IETF, June 2022. Available: <https://www.rfc-editor.org/rfc/rfc9110>
- [2] OWASP Foundation, OWASP Top 10 API Security Risks – 2023. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [3] OWASP Foundation, API8:2023 Security Misconfiguration. Available: <https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/>
- [4] M. Souppaya, K. Scarfone, and D. Dodson, Secure Software Development Framework (SSDF) Version 1.1, NIST SP 800-218, Feb. 2022. Available: <https://doi.org/10.6028/NIST.SP.800-218>
- [5] Rentgen, Project Repository and Releases. Available: <https://github.com/Rentgen-io/Rentgen>
- [6] Rentgen, API Stories and Case Studies. Available: <https://rentgen.io/api-stories>
- [7] L. Jankauskas, Automation Before Automation (ABA): An Intermediate Phase in Modern Software Testing, 2026. Available: <https://gaontime.com/research/automation-before-automation.html>
- [8] Rentgen.io Research Initiative, Boruzele: AI-Powered Exploratory Testing Agent, 2026. Available: <https://github.com/Rentgen-io/boruzele>